RETRY DESIGN

My scheduler retried one post 298 times

The date was already in the past on every attempt. The log shows what a retry policy without a classification does.

ENTER



EXHIBIT 02 · THE LOG

298 attempts. A date in the past never becomes the future.

One old post failed 298 times with 'dueAt must be in the future', another 229 times. Every attempt re-uploaded the media to GitHub first. The log has no timestamps.

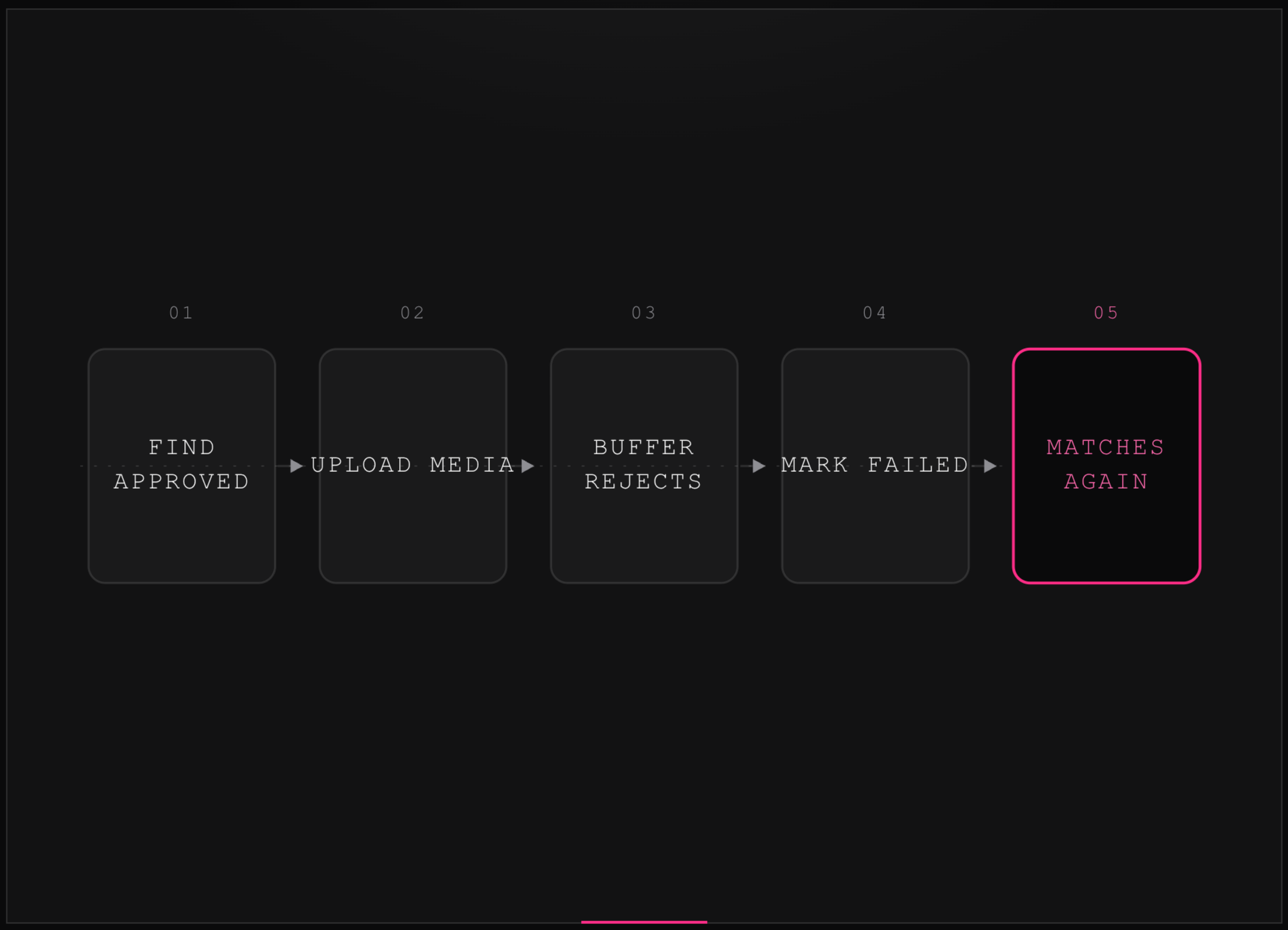


EXHIBIT 03 · THE LOOP

A Failed post matches the same query on the next run

The query takes Approved and not Scheduled. Failed is not Scheduled. No attempt counter, no cap, no back-off. The process exits 0 whatever happened.

HARMLESS TO RETRY		PERMANENT
— BAD GITHUB TOKEN	VS	• DATE IN THE PAST
— FAILS BEFORE BUFFER		• PARK, DON'T RETRY

• EXHIBIT 04 · CLASSIFICATION

Retry the transient. Park the permanent.

A bad GitHub token failed before Buffer was called, so those retries were harmless. A past date never recovers. The loop treated both the same.

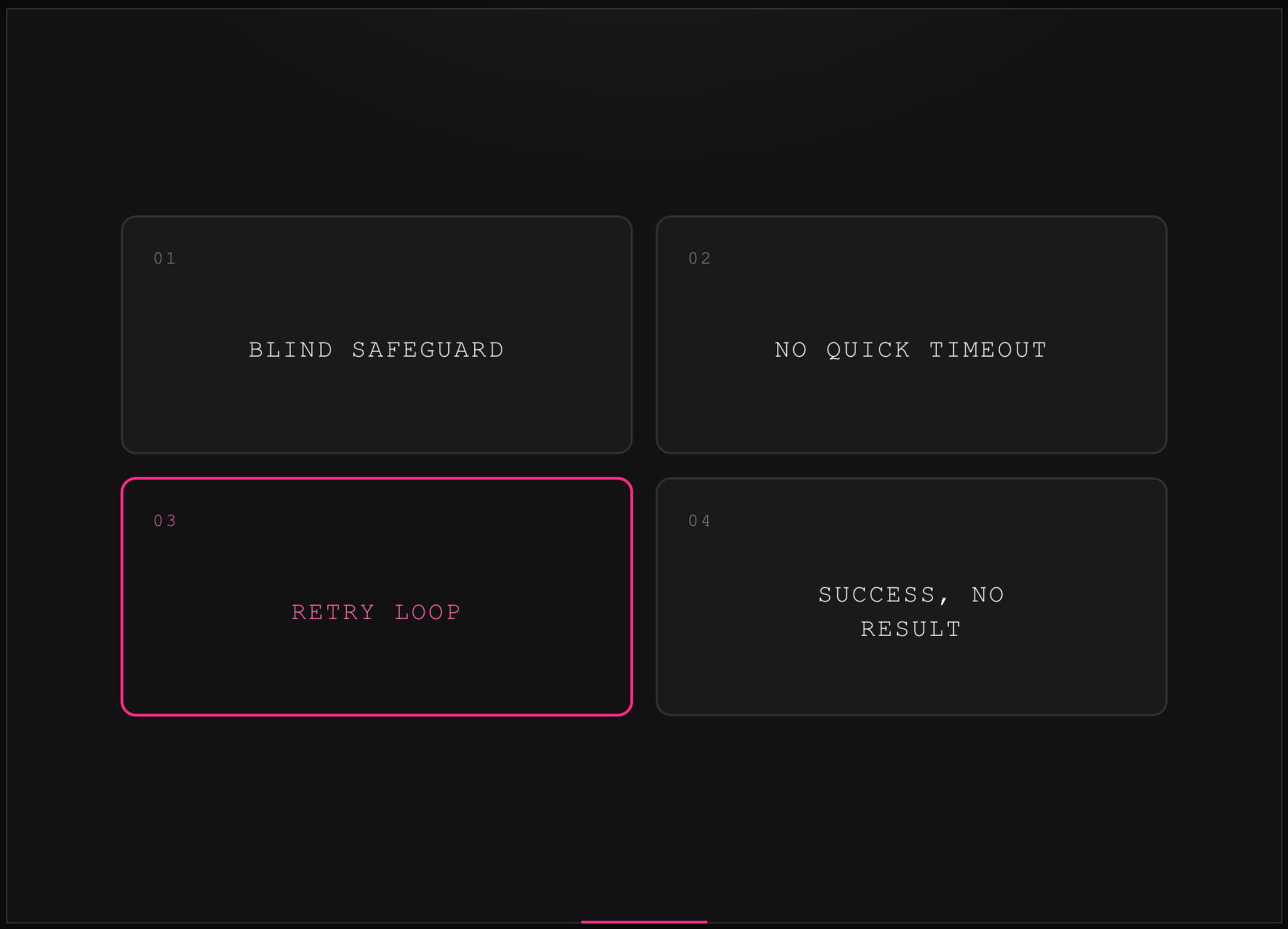


EXHIBIT 05 · THE POSTMORTEM

GitHub's September report lists four failure factors

From GitHub's October 14 report on two September incidents: a safeguard blind to primary load, no quick timeout, a retry loop on failing writes, and a job reporting success with no review.

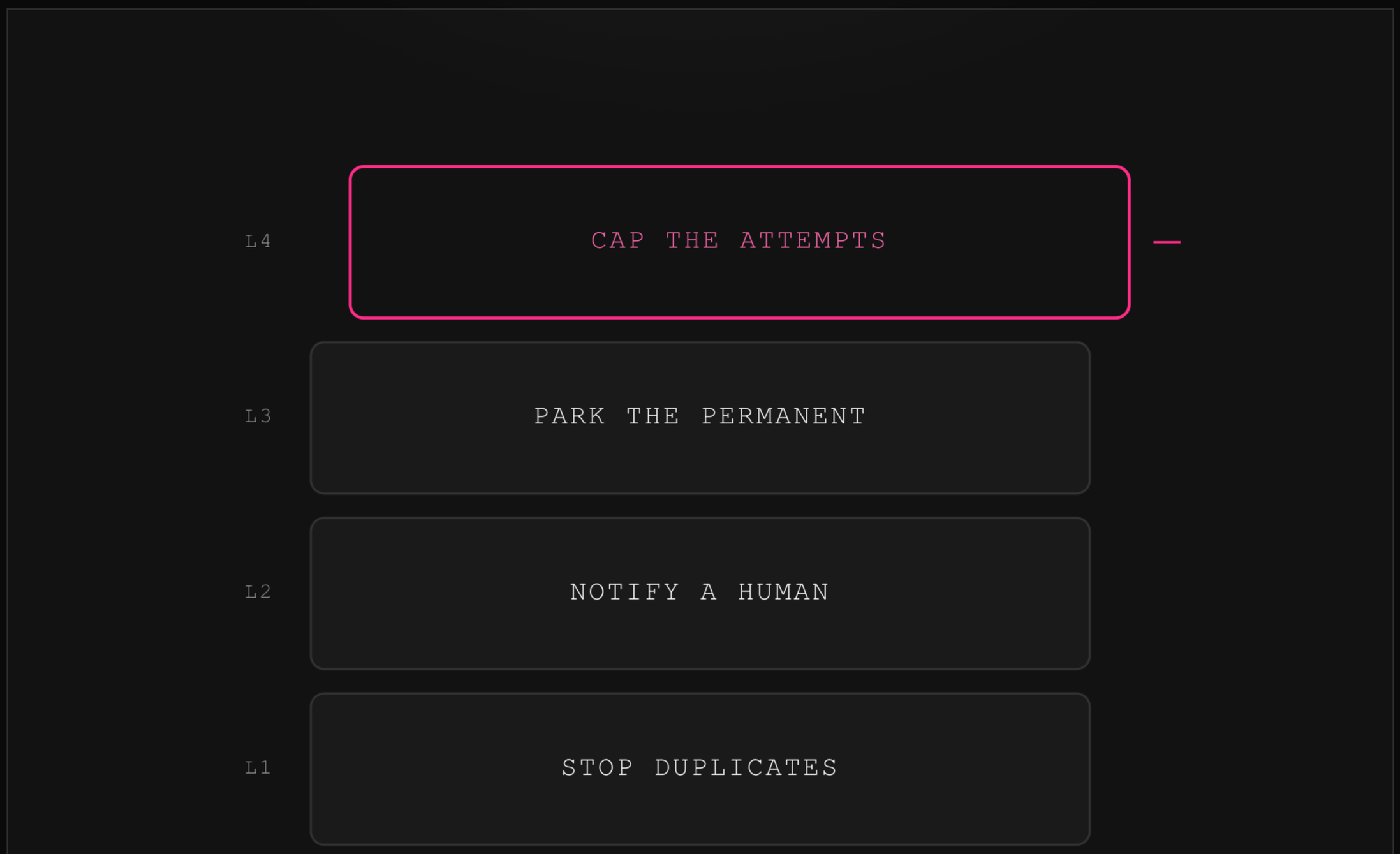


EXHIBIT 06 · THE FIX

What I would change in the scheduler

Cap the attempts. Park permanent errors instead of retrying them. Notify me either way.

From reading the code: stop a failed Notion update from causing a second Buffer post.



Does your retry loop know the difference?

Classify the error first, then decide whether to retry.

AUDIT YOUR RETRIES